Ders 1 Çalışma Özeti

Ders 1 Çalışma Özeti

Genel Konular

- Asıl ders yaklaşık 00:08'de başlar. Öncesi bekleme ve sohbettir.
- Dersin öncesi ve sonrası
  - Öncesi: yapısal programlama ve algoritma bilgisi (C, Pascal).
  - Sonrası: veri tabanı, sistem analizi, web ve mobil programlama. Nesne tabanlı yaklaşım bunların temelidir.
- Nesne yönelimli programlamanın (object-oriented programming) amacı
  - Yapısal programlamada program fonksiyonlara ve dosyalara bölünür. Büyük projeyi modüler ve yönetilebilir tutmak için bu yetmez.
  - Nesne yönelimli yaklaşım; zaman, yönetim, bakım ve anlaşılabilirlik açısından kazanç sağlar.
  - Yazılımın "yaşayabilir" olması gerekir: Gereksinim değişince sistemi kapatmadan, büyük değişiklik yapmadan yeni modül eklenebilmelidir.
- Nesne tabanlı analiz ve tasarım
  - Kodlamadan önce problemdeki nesneler ve aralarındaki ilişkiler belirlenir.
  - Geliştirme adımları: problem, analiz, tasarım, gerçekleme (implementation), test ve bakım, güncelleme.
  - Modelleme dili: UML (Unified Modeling Language).
- Dönem planı
  - Java programlamaya giriş, nesne ve sınıf (object, class), UML sınıf şemaları.
  - İlkel tipler (primitive) ve sarmalayıcı sınıflar (wrapper), parametre geçişi.
  - Nesneler arası ilişkiler, UML etkileşim (sequence) diyagramları.
  - Kalıtım (inheritance), soyut sınıf (abstract class), arayüz (interface), çok biçimlilik (polymorphism), override ve overload.
  - Enumeration, jenerik sınıflar (generics), istisna yönetimi (exception handling), tip dönüşümleri.
  - Dosyalar ve akışlar (stream), çoklu iş parçacığı (multithreading), iç sınıflar (inner class), grafik arayüz (GUI) ve olay güdümlü programlama (event-driven programming).
- Java'ya giriş
  - Java; masaüstünde, web'de ve küçük cihazlarda çalışabilen taşınabilir kod üretmek için yaygınlaştı.
  - Sürümler (edition): Standard Edition (SE), Micro Edition (ME), Enterprise Edition (EE).
  - Derleme ve çalıştırma: `.java` kaynak kodu Java derleyicisiyle `.class` uzantılı bytecode'a çevrilir. Bytecode, Java Virtual Machine (JVM) tarafından çalıştırılır.
  - JDK (Java Development Kit) geliştirici içindir. JRE (Java Runtime Environment) programı çalıştırmak içindir. JDK, JRE'yi de içerir.
  - IDE olarak hoca NetBeans kullanıyor. Eclipse ve IntelliJ de kullanılabilir.
  - UML araçları: Violet UML, ArgoUML, NetBeans ve Eclipse eklentileri (plugin).

Hocanın Özellikle Vurguladığı Kısımlar

- Bu dersin amacı Java dilini öğrenmek değil.
  - Asıl zor olan nesne tabanlı analiz ve tasarımdır: Hangi sınıflar olacak, nesneler arasındaki ilişkiler nasıl olacak?

  - Piyasa, nesne yönelimli mantığı bilmeyen mühendisi zorlanmış görüyor.
- Analizde yapısal düşünmeyin.
  - Örnek: Kitabı dört ayrı değişkenle (id, ad, sayfa sayısı, yazar adı) tutarsanız, sonradan "kitap türü" alanı eklemek bütün kodu değiştirmeyi gerektirir. Kitap nesnesi olarak tasarlarsanız sadece bir alan eklersiniz.
- Kaynak okuyun.
  - Martin Fowler, "UML Distilled"; Fowler'ın "Refactoring" ve "Analysis Patterns" kitapları.
  - Daniel Liang, "Introduction to Java Programming"; Deitel, "Java How to Program"; "Core Java"; "Absolute Java".
  - Türkçe: Özcan Acar, "Java Tasarım Şablonları ve Yazılım Mimarileri"; aynı yayınevinden "Tasarım Desenleri ve Mimarileri (C#, Java, C++ Örnekleri)"; Aykut Taşdelen'in (altyazıda "Aykut Taşderen") C++, Java ve C# ile UML ve tasarım şablonları kitabı.
  - Hoca ayrıca Türkçe bir Java kitabını güzel bulduğunu söyledi.
  - Bunlar sınav için değil, meslek için önerildi.
- Uzaktan eğitimin avantajını kullanın.
  - Anlamadığınız konuyu kayıttan veya diğer grupların hocalarının kayıtlarından tekrar izleyin.
- Veri yapıları dersindeki konuları Java'da da yazın. Hem Java hem veri yapıları bilginiz gelişir.

Kısa Tekrar Notları

- Dersin iki tarafı var: nesneye dayalı kavramlar (teori, UML) ve nesneye dayalı programlama (Java).
- Nesne tabanlı yaklaşımın kazançları: zaman, yönetim, bakım, anlaşılabilirlik.
- Geliştirme adımları: problem, analiz, tasarım, gerçekleme, test ve bakım.
- UML, nesne yönelimli tasarımın ortak modelleme dilidir. UML araçlarının bir kısmı modelden kod iskeleti üretir (code generation).
- Java'da işaretçi (pointer) doğrudan yoktur. Ama nesnelere referans (reference) üzerinden erişilir; bellekte referans mantığını hep aklınızda tutun.
- Java'da operatör aşırı yükleme (operator overloading) ve çoklu kalıtım (multiple inheritance) yoktur. Çoklu kalıtıma benzer davranış arayüzlerle (interface) sağlanır.
- Java 8'den itibaren arayüzlerde `default` anahtar kelimesiyle gövdeli metot yazılabilir.
- `.java` dosyası derleyiciyle `.class` (bytecode) dosyasına çevrilir. JVM bytecode'u çalıştırır.
- JDK: geliştirici için. JRE: çalıştırmak için.
- Java sürümleri: SE (masaüstü, genel), ME (küçük cihazlar), EE (kurumsal, web ve sunucu uygulamaları).
- Çok katmanlı uygulama: veri tabanı katmanı, iş mantığı (business logic) katmanı, kullanıcı arayüzü katmanı.
- Grafik arayüz kütüphaneleri tarihsel sırayla: AWT, Swing, JavaFX.
- İstisna (exception): çalışma zamanında oluşan hata durumu. Örnek: sıfıra bölme, dizi sınırı dışına erişim. Sözdizimi hatası (syntax error) istisna değildir.

Detaylı Açıklamalar

Nesne yönelimli yaklaşım neden gerekli?

- Yapısal programlamada büyük bir projede bir değişkenin türünü değiştirmek, kodun birçok yerini değiştirmeyi gerektirebilir.
- C ve Pascal donanıma yakın, soyutlamaya uzak dillerdir. Nesne yönelimli tasarımda soyut düşünülür.
- Hoca, yeni modül eklemeyi robota parça takmaya benzetti: Yeni parça takılır, sistem uyum sağlar, yeni işlevler ortaya çıkar.
- Bir web sitesinin sayfaları nesneler üzerinde işlem yapar: kitap ekleme, silme, yazar ekleme. Kitap ile yazar arasındaki ilişki ayrıca tasarlanır.

Nesneler arası ilişkiler ve tasarım şablonları

- İlerleyen haftalarda görülecek ilişkiler: kalıtım, çok biçimlilik, bileşim (composition), toplama (aggregation).
- Bazı problemler için bu kavramlar yetmez. O zaman tasarım şablonları (design patterns) kullanılır. Bu konu ayrı bir ders konusudur.
- UML'de 12-13 diyagram türü var. Bu derste ağırlık sınıf diyagramlarındadır. Kullanım durumu (use case) diyagramı gibi türler sistem analizi ve yazılım mühendisliği derslerinde görülür.
  - Use case örneği: Hastane yazılımında doktor, hasta ve yönetimin rolleri farklıdır. Bu roller use case diyagramıyla gösterilir.

Çok biçimlilik için İstanbulkart örneği

- Öğrenci, öğretmen ve emekli birer kullanıcıdır.
- Kart okutulunca her birinin bakiyesinden farklı miktar düşer.
- Aynı işlemin kullanıcı türüne göre farklı davranması çok biçimliliğe ve metotların override edilmesine örnektir.

Jenerik sınıflar

- Bir diziyi sıralayan sınıf yazdınız. Dizi `int`, `double`, `String` ya da kitap nesnesi olabilir.
- Her tip için ayrı sınıf yazmak yerine jenerik (generic) sınıf yazılır.

İstisna yönetimi

- Programda istisna oluşunca programın kapanmasını istemeyiz.
- C'de yapılan şey istisnadan kaçınmaktır (exception avoidance): Örneğin bölen sıfırsa `do-while` ile veriyi tekrar isteriz.
- Java'da istisna oluşunca onu yakalayıp işleyen bir mekanizma kurulur. Kendi istisna türlerimizi de tanımlayabiliriz.
- Hoca bunu Assembly'deki kesmeye (interrupt) benzetti: Olayın ne zaman olacağı bilinmez, olunca ona bağlı işlem çalışır.
- Dizi sınırı dışına erişim: C'de 10 elemanlı dizide `a[10]`'a erişmek çoğu zaman hata vermez. Java'da çalışma zamanında `ArrayIndexOutOfBoundsException` oluşur.

Java'nın derlenmesi ve çalıştırılması

- Kaynak kod: `.java` uzantılı dosya. Sözdizimi C'ye çok benzer.
- Java derleyicisi (`javac`) kaynak kodu `.class` uzantılı bytecode'a çevirir. Bytecode not defteriyle açılınca anlaşılmaz baytlar görülür.
- Aynı `.class` dosyası, JVM bulunan her işletim sisteminde çalışır. Kullanılan kütüphanenin o platformda desteklenmesi gerekir.
- JVM, bytecode'u bulunduğu işletim sistemine ve işlemciye göre çalıştırır.
- Komut satırından da derlenip çalıştırılabilir. Ama IDE bu adımları birlikte yapar ve hata ayıklama (debug) imkânı verir.

Java sürümleri ve kurumsal uygulamalar

- Standard Edition: Komut satırı ve masaüstü uygulamaları. Derste bu kullanılacak.
- Micro Edition: Küçük cihazlar için.
- Enterprise Edition: İstemci-sunucu (client-server) mimarisi, web servisleri, arka uç (backend). Spring Framework, Apache Tomcat, IBM WebSphere örnek verildi.
- E-Devlet örneği: Öğrenci belgesi alırken E-Devlet, üniversitenin sunucusundan servis alır. Servisler arası bu tür işlemler kurumsal uygulamalarda yapılır.
- Java; Oracle'ın sağladığı kapsamlı dokümantasyonu sayesinde öğrenmesi ve kullanması kolay bir dildir.
- Java her 6 ayda bir yeni sürüm çıkarır. Bu yüzden bazı eski bilgiler zamanla geçersiz olur (örnek: arayüzlerde `default` metot).

Plugin kavramı

- Bir uygulamaya sonradan yeni özellik eklemek için bırakılan "açık kapıdır". Bu bir tasarım kalıbıdır.
- NetBeans ve Eclipse'te UML çizimi gibi işlevler plugin ile eklenir.
- Ders sonunda bir öğrenci ekran paylaşıp IntelliJ'i tanıttı. NetBeans plugin listesinde "PlantUML Integration" ve "Simple UML" gibi eklentiler bulundu.

Ders sonrası notlar

- **Java'da parametre geçişi:** Hoca "C'de gerçek call by reference yok, Java'da gerçek bir call by reference mekanizması var" dedi. İlk kısım doğru. Ancak Java'da da parametreler her zaman **değerle** (call by value) geçer. Nesne parametresinde kopyalanan değer referanstır. Bu yüzden metot içinde nesnenin alanı değiştirilebilir, ama parametreye yeni nesne atamak çağıranın değişkenini değiştirmez. Java ile denendi:

```
static void degistir(Kitap k) {
    k.ad = "Degisti";          // çağıranın nesnesi değişir
    k = new Kitap(99, "Yeni"); // çağıranın değişkeni değişmez
}
```

Hoca swap için wrapper sınıf gerektiğini de söyledi. `Integer` parametreleri içeride takas etmek de çağırandaki değerleri değiştirmez: Java ile denendi, `swap(a, b)` sonrası `a=1, b=2` kaldı.

- **Derleyici ve JVM:** Hoca derleyicinin "yorumlayıcıyı da içerdiğini" ve JVM'nin `.jar` uzantılı "executable dosya oluşturduğunu" söyledi. Doğrusu: `javac` yalnızca derler. JVM bytecode'u yorumlar ve sık kullanılan kısımları çalışma anında makine koduna çevirir (JIT derleme). `.jar` dosyası JVM'nin ürettiği bir şey değildir; `.class` dosyalarını ve kaynakları bir arada tutan bir arşivdir (`jar` aracıyla oluşturulur).
- **Dizi sınırı:** Hoca Java'nın `a[10]` erişimini "derlerken doğrudan hata verdiğini" söyledi. Java derleyicisi dizi sınırını derleme zamanında denetlemez. Hata çalışma zamanında `ArrayIndexOutOfBoundsException` olarak gelir.
- **Çoklu kalıtım:** Java'nın güncel sürümlerinde de sınıflar arası çoklu kalıtım yoktur. Bir sınıf birden fazla arayüzü uygulayabilir (`implements`).
- **Java sürümü:** Ders tarihinde (10 Mart 2021) güncel sürüm Java SE 15'ti. Java SE 16, 16 Mart 2021'de yayımlandı.
- **Java ME ve Android:** Android uygulamaları Java ile yazılır ama Java ME kullanmaz. Android'in kendi SDK'sı vardır.
- **Yazar adları:** Kitap ve yazar adları altyazıdan alındı. Satın almadan veya aramadan önce adları kontrol edin.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.